B+ Trees Implementation

Laksh Jain

What are B+ Trees

> A self balancing binary search tree
> Data is stored only in the leaves
> Leaves form a linked list
> O(logn)

Structure

Order (m): Max number of children

Max Keys = m-1

Min Keys = m//2 - 1

```python
class BPlusTreeNode:
    def __init__(self, order, is_leaf=False):
        self.order = order
        self.is_leaf = is_leaf
        self.keys = []
        self.children = []
        self.values = []  # Separate array for leaf nodes values
        self.next = None

    def is_full(self):
        return len(self.keys) == self.order - 1

class BPlusTree:
    def __init__(self, order=8):
        self.order = order
        self.root = BPlusTreeNode(order, is_leaf=True)
```

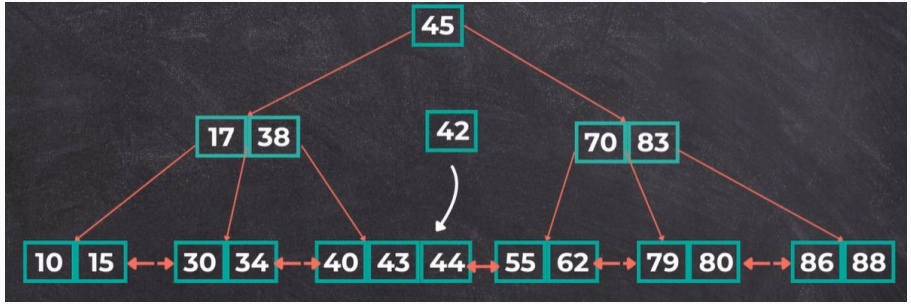
Search

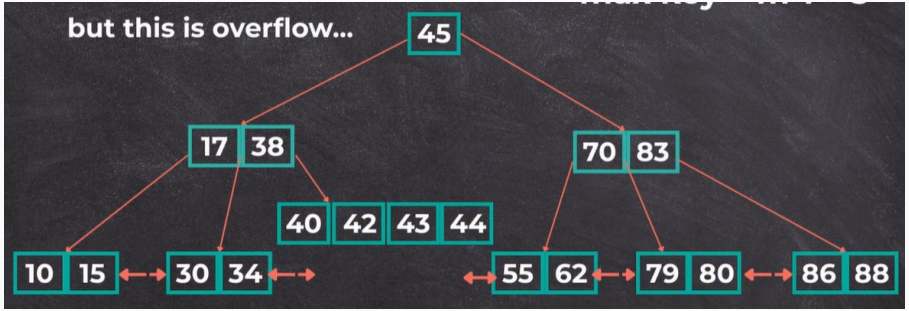
```python
def search(self, key):
    node = self.root
    while not node.is_leaf:
        i = bisect.bisect_right(node.keys, key)
        node = node.children[i]

    i = bisect.bisect_left(node.keys, key)
    if i < len(node.keys) and node.keys[i] == key:
        return node.values[i]
    return None
```

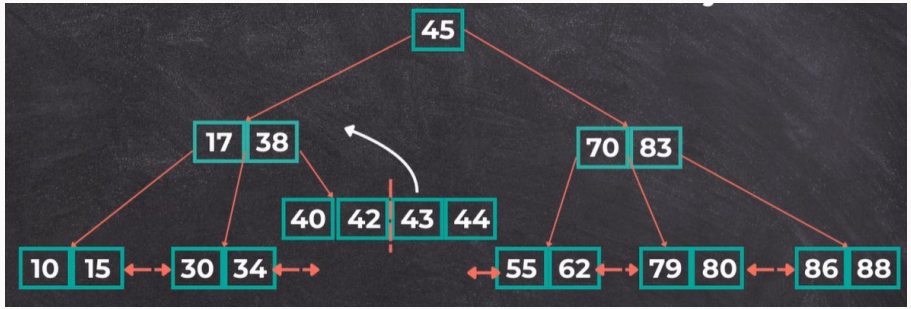
Range Queries

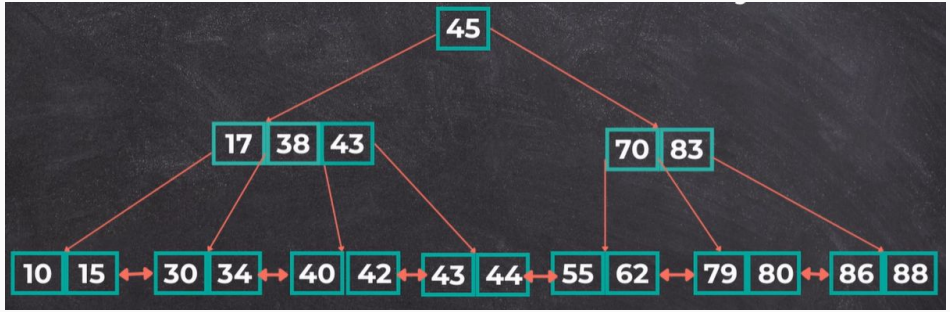
```python
def range_query(self, start_key, end_key):
    node = self.root
    while not node.is_leaf:
        i = bisect.bisect_right(node.keys, start_key)
        node = node.children[i]

    results = []
    while node:
        for k, v in zip(node.keys, node.values):
            if start_key <= k <= end_key:
                results.append((k, v))
            elif k > end_key:
                return results
        node = node.next
    return results
```

Insertion


45
17 38
42
70 83
10 15 30 34 40 43 44 55 62 79 80 86 88


but this is overflow...
45
17 38
70 83
40 42 43 44
10 15 30 34 55 62 79 80 86 88


45
17 38
70 83
40 42 43 44
10 15 30 34 55 62 79 80 86 88


45
17 38 43
70 83
10 15 30 34 40 42 43 44 55 62 79 80 86 88

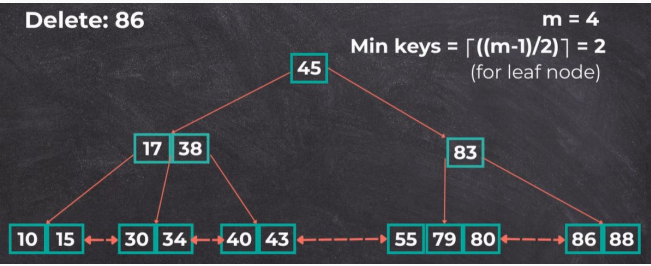
Insertion

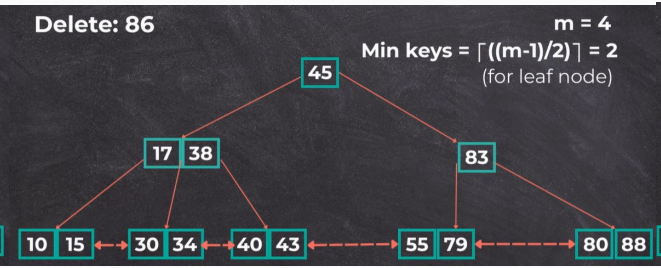
```python
def insert(self, key, value):
    root = self.root
    if root.is_full():
        new_root = BPlusTreeNode(self.order, is_leaf=False)
        self.root = new_root
        new_root.children.append(root)
        self._split_child(new_root, 0)
        self._insert_non_full(new_root, key, value)
    else:
        self._insert_non_full(root, key, value)
```

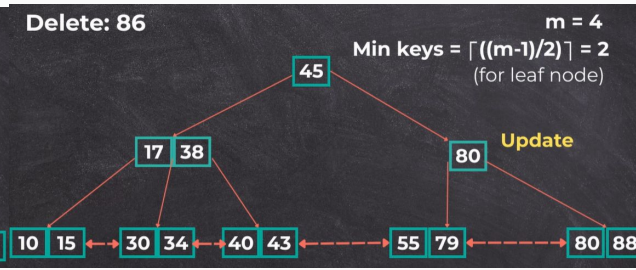
```python
def _insert_non_full(self, node, key, value):
    if node.is_leaf:
        i = bisect.bisect_left(node.keys, key)
        if i < len(node.keys) and node.keys[i] == key:
            node.values[i] = value
        else:
            node.keys.insert(i, key)
            node.values.insert(i, value)
    else:
        i = bisect.bisect_right(node.keys, key)
        if node.children[i].is_full():
            self._split_child(node, i)
            if key > node.keys[i]:
                i += 1
        self._insert_non_full(node.children[i], key, value)
```

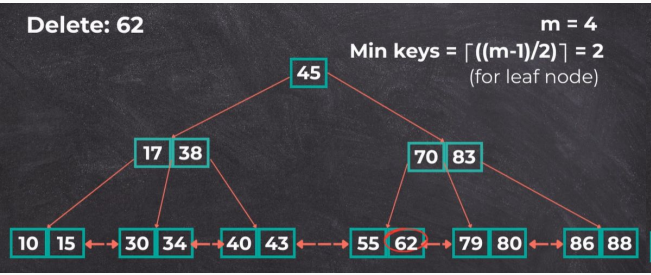
```python
def _split_child(self, parent, index):
    order = self.order
    node = parent.children[index]
    new_node = BPlusTreeNode(order, is_leaf=node.is_leaf)

    if node.is_leaf:
        mid = len(node.keys) // 2
        new_node.keys = node.keys[mid:]
        new_node.values = node.values[mid:]
        node.keys = node.keys[:mid]
        node.values = node.values[:mid]

        new_node.next = node.next
        node.next = new_node

        parent.keys.insert(index, new_node.keys[0])
        parent.children.insert(index + 1, new_node)
    else:
        mid = len(node.keys) // 2
        new_node.keys = node.keys[mid+1:]
        new_node.children = node.children[mid+1:]
        up_key = node.keys[mid]
        node.keys = node.keys[:mid]
        node.children = node.children[:mid+1]

        parent.keys.insert(index, up_key)
        parent.children.insert(index + 1, new_node)
```

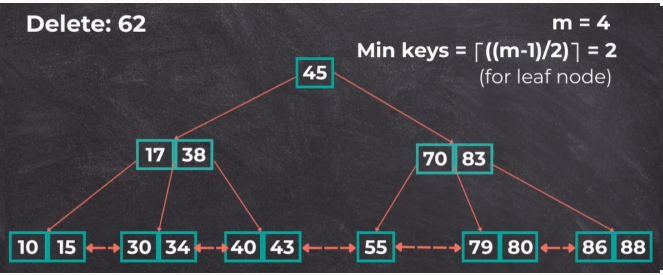
Deletion

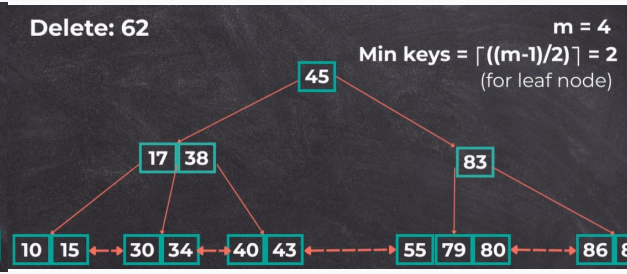

Delete: 86
m = 4
Min keys = ⌈((m-1)/2)⌉ = 2
(for leaf node)
45
17 38
83
10 15 30 34 40 43 55 79 80 86 88


Delete: 86
m = 4
Min keys = ⌈((m-1)/2)⌉ = 2
(for leaf node)
45
17 38
83
10 15 30 34 40 43 55 79 80 88


Delete: 86
m = 4
Min keys = ⌈((m-1)/2)⌉ = 2
(for leaf node)
Update
45
17 38
80
10 15 30 34 40 43 55 79 80 88


Delete: 62
m = 4
Min keys = ⌈((m-1)/2)⌉ = 2
(for leaf node)
45
17 38
70 83
10 15 30 34 40 43 55 62 79 80 86 88


Delete: 62
m = 4
Min keys = ⌈((m-1)/2)⌉ = 2
(for leaf node)
45
17 38
70 83
10 15 30 34 40 43 55 79 80 86 88


Delete: 62
m = 4
Min keys = ⌈((m-1)/2)⌉ = 2
(for leaf node)
45
17 38
83
10 15 30 34 40 43 55 79 80 86

Deletion

```python
def delete(self, key):
    if not self.root.keys:
        return False

    deleted = self._delete(self.root, key)

    if not self.root.keys and not self.root.is_leaf:
        self.root = self.root.children[0]

    return deleted

def _delete(self, node, key):
    min_keys = (self.order // 2) - 1 if self.order % 2 == 0 else self.order // 2

    if node.is_leaf:
        i = bisect.bisect_left(node.keys, key)
        if i < len(node.keys) and node.keys[i] == key:
            node.keys.pop(i)
            node.values.pop(i)
            return True
        return False
    else:
        i = bisect.bisect_right(node.keys, key)
        deleted = self._delete(node.children[i], key)

        if deleted:
            if len(node.children[i].keys) < min_keys:
                self._fill_child(node, i)

        return deleted
```

Deletion

```python
def _fill_child(self, node, index):
    min_keys = (self.order // 2) - 1 if self.order % 2 == 0 else self.order // 2

    if index > 0 and len(node.children[index - 1].keys) > min_keys:
        self._borrow_from_prev(node, index)
    elif index < len(node.children) - 1 and len(node.children[index + 1].keys) > min_keys:
        self._borrow_from_next(node, index)
    else:
        if index < len(node.children) - 1:
            self._merge(node, index)
        else:
            self._merge(node, index - 1)
```

```python
def _borrow_from_prev(self, node, index):
    child = node.children[index]
    sibling = node.children[index - 1]

    if child.is_leaf:
        child.keys.insert(0, sibling.keys.pop(-1))
        child.values.insert(0, sibling.values.pop(-1))
        node.keys[index - 1] = child.keys[0]
    else:
        child.keys.insert(0, node.keys[index - 1])
        node.keys[index - 1] = sibling.keys.pop(-1)
        child.children.insert(0, sibling.children.pop(-1))
```

```python
def _borrow_from_next(self, node, index):
    child = node.children[index]
    sibling = node.children[index + 1]

    if child.is_leaf:
        child.keys.append(sibling.keys.pop(0))
        child.values.append(sibling.values.pop(0))
        node.keys[index] = sibling.keys[0]
    else:
        child.keys.append(node.keys[index])
        node.keys[index] = sibling.keys.pop(0)
        child.children.append(sibling.children.pop(0))
```

```python
def _merge(self, node, index):
    child = node.children[index]
    sibling = node.children[index + 1]

    if child.is_leaf:
        child.keys.extend(sibling.keys)
        child.values.extend(sibling.values)
        child.next = sibling.next
        node.keys.pop(index)
        node.children.pop(index + 1)
    else:
        child.keys.append(node.keys.pop(index))
        child.keys.extend(sibling.keys)
        child.children.extend(sibling.children)
        node.children.pop(index + 1)
```

Update

```python
def update(self, key, new_value):
    node = self.root
    while not node.is_leaf:
        i = bisect.bisect_right(node.keys, key)
        node = node.children[i]

    i = bisect.bisect_left(node.keys, key)
    if i < len(node.keys) and node.keys[i] == key:
        node.values[i] = new_value
        return True
    return False
```